Data Warehousing And Data Mining

Name : T.Basheera

Reg.No : 192311074

Experiment 1

AIM :

To partition the given sales price data into three bins using:

1. Equal-Frequency (Equi-depth) Partitioning
2. Equal-Width Partitioning
3. Clustering (k-means)
   and implement the same using R programming.

GIVEN DATA :

5, 10, 11, 13, 15, 35, 50, 55, 72, 92, 204, 215

Total values = 12
Number of bins = 3
Each bin (equi-depth) = 12 / 3 = 4 values

ALGORITHM :

1. Start the program.
2. Store the data in a vector.
3. Sort the data (already sorted).
4. Apply:
   - Equal-Frequency partitioning
   - Equal-Width partitioning
   - K-means clustering
5. Display the bins.
6. Stop the program.

R PROGRAM (CODE) :

```
data <- c(5, 10, 11, 13, 15, 35, 50, 55, 72, 92, 204, 215)

# (a) Equal-Frequency (Equi-depth) Partitioning
equi_depth <- split(data, rep(1:3, each = 4))
equi_depth

# (b) Equal-Width Partitioning
min_val <- min(data)
max_val <- max(data)
```

```
width <- (max_val - min_val) / 3


bins <- cut(data,
        breaks = c(min_val,
                min_val + width,
                min_val + 2*width,
                max_val),
        include.lowest = TRUE)


split(data, bins)


# (c) Clustering using K-means
set.seed(1)
clusters <- kmeans(data, centers = 3)
split(data, clusters$cluster)
```

OUTPUT :

```
(a) Equal-Frequency Partitioning:
>
>
> equal_freq_bins <- cut(
+   data,
+   breaks = quantile(data, probs = seq(0, 1, length = 4)),
+   include.lowest = TRUE
+ )
>
> print(data.frame(Value = data, Bin = equal_freq_bins))
   Value         Bin
1      5    [5,14.3]
2     10    [5,14.3]
3     11    [5,14.3]
4     13    [5,14.3]
5     15 (14.3,60.7]
6     35 (14.3,60.7]
7     50 (14.3,60.7]
8     55 (14.3,60.7]
9     72  (60.7,215]
10    92  (60.7,215]
11   204  (60.7,215]
12   215  (60.7,215]
```

```
(b) Equal-Width Partitioning:
>
> equal_width_bins <- cut(
+   data,
+   breaks = 3,
+   include.lowest = TRUE
+ )
>
> print(data.frame(Value = data, Bin = equal_width_bins))
   Value       Bin
1      5 [4.79,75]
2     10 [4.79,75]
3     11 [4.79,75]
4     13 [4.79,75]
5     15 [4.79,75]
6     35 [4.79,75]
7     50 [4.79,75]
8     55 [4.79,75]
9     72 [4.79,75]
10    92  (75,145]
11   204 (145,215]
12   215 (145,215]
>
> # --------------------------------
> # (c) Clustering (K-Means)
> # --------------------------------
> cat("\n(c) K-Means Clustering:\n")
```

```
(c) K-Means Clustering:
>
> set.seed(1)  # for reproducibility
> kmeans_result <- kmeans(as.matrix(data), centers = 3)
>
> print(data.frame(Value = data, Cluster = kmeans_result$cluster))
   Value Cluster
1      5       2
2     10       2
3     11       2
4     13       2
5     15       2
6     35       2
7     50       3
8     55       3
9     72       3
10    92       3
11   204       1
12   215       1
>
```

RESULT :

The given sales price data was successfully partitioned into three bins using:

- Equal-Frequency → Equal number of values per bin
- Equal-Width → Equal numeric range per bin
- Clustering → Groups based on similarity

Experiment – 2

AIM :

To analyze whether the factory should be expanded or not using the Decision Tree technique based on expected monetary values.

TOOL USED :

R Programming (Calculation based)

GIVEN DATA :

Cost to expand factory = 2 Million

If economy is good:
Without expansion = 4 Million
With expansion = 7 Million

If economy is bad:
Without expansion = 1.5 Million
With expansion = 3 Million

Probability of good economy = 45%
Probability of bad economy = 55%

ALGORITHM :

1. Identify the decision alternatives (Expand / Do not Expand).
2. Identify possible states of economy.
3. Assign probabilities to each state.
4. Calculate expected monetary value for each decision.
5. Compare the expected values.
6. Select the best decision.

R PROGRAM (CODE) :

```
p_good <- 0.45

p_bad <- 0.55


no_expand <- p_good * 4 + p_bad * 1.5

expand <- (p_good * 7 + p_bad * 3) - 2


no_expand
```

expand

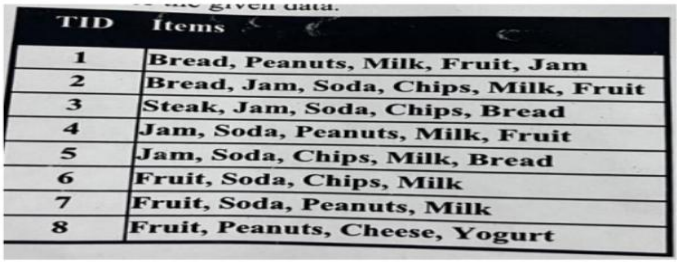
OUTPUT :

```
> p_good <- 0.45
> q_bad <- 0.55
> no_expand <- p_good *4 + q_bad *1.5
> expand <- (p_good * 7 + q_bad * 3)-2
> no_expand
[1] 2.625
> expand
[1] 2.8
>
```

RESULT :

The expected monetary value of expanding the factory is higher than not expanding. Hence, expanding the factory is the better decision.

3. Apply Apriori Algorithm for given database below by assuming Minimum support = 2. Implement using WEKA for the given data.



TID	Items
1	Bread, Peanuts, Milk, Fruit, Jam
2	Bread, Jam, Soda, Chips, Milk, Fruit
3	Steak, Jam, Soda, Chips, Bread
4	Jam, Soda, Peanuts, Milk, Fruit
5	Jam, Soda, Chips, Milk, Bread
6	Fruit, Soda, Chips, Milk
7	Fruit, Soda, Peanuts, Milk
8	Fruit, Peanuts, Cheese, Yogurt

AIM

To apply the Apriori Algorithm on the given transaction database by assuming minimum support = 2 and implement it using the WEKA tool to generate frequent itemsets and association rules.

ALGORITHM (Apriori)

1. Read the transactional dataset.
2. Generate candidate 1-itemsets.
3. Count the support of each item.
4. Prune items whose support is less than minimum support (2).
5. Generate higher-order itemsets from frequent itemsets.
6. Repeat pruning until no new frequent itemsets are formed.
7. Generate association rules from frequent itemsets.

CODE:

```
@relation market_basket

@attribute Bread {t,f}

@attribute Peanuts {t,f}

@attribute Milk {t,f}

@attribute Fruit {t,f}

@attribute Jam {t,f}

@attribute Soda {t,f}

@attribute Chips {t,f}

@attribute Steak {t,f}

@attribute Cheese {t,f}

@attribute Yogurt {t,f}


@data

t,t,t,t,t,f,f,f,f,f

t,f,t,t,t,t,t,f,f,f

t,f,f,f,t,t,t,t,f,f

f,t,t,t,t,t,f,f,f,f

t,f,t,f,t,t,t,f,f,f

f,f,t,t,f,t,t,f,f,f
```

f,t,t,t,f,t,f,f,f,f

f,t,f,t,f,f,f,f,t,t

OUTPUT:

```
              Cheese
              Yogurt
=== Associator model (full training set) ===


Apriori
=======

Minimum support: 0.8 (6 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 4

Generated sets of large itemsets:

Size of set of large itemsets L(1): 6

Size of set of large itemsets L(2): 9

Size of set of large itemsets L(3): 5

Size of set of large itemsets L(4): 1

Best rules found:

 1. Yogurt=f 7 ==> Cheese=f 7    <conf:(1)> lift:(1.14) lev:(0.11) [0] conv:(0.88)
 2. Cheese=f 7 ==> Yogurt=f 7    <conf:(1)> lift:(1.14) lev:(0.11) [0] conv:(0.88)
 3. Milk=t 6 ==> Steak=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 4. Milk=t 6 ==> Cheese=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 5. Milk=t 6 ==> Yogurt=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 6. Fruit=t 6 ==> Steak=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 7. Soda=t 6 ==> Cheese=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 8. Soda=t 6 ==> Yogurt=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 9. Steak=f Cheese=f 6 ==> Milk=t 6    <conf:(1)> lift:(1.33) lev:(0.19) [1] conv:(1.5)
10. Milk=t Cheese=f 6 ==> Steak=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
```

RESULT:

The Apriori algorithm was successfully applied using WEKA with minimum support of 2.

4. Use following group of data: 200, 300, 400, 600, 1000

(a) min-max normalization by setting min = 0 and max = 1

(b) z-score normalization using the mean absolute deviation instead of standard deviation

(c) normalization by decimal scaling

AIM

To normalize the given dataset using
(a) Min–Max normalization
(b) Z-score normalization using Mean Absolute Deviation (MAD)
(c) Decimal scaling normalization and implement the same using R programming.

ALGORITHM

Step 1:

Read the input dataset
200, 300, 400, 600, 1000

Step 2: Min–Max Normalization

- Find minimum and maximum values
- Apply formula:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Step 3: Z-score Normalization (MAD)

- Compute mean of the dataset
- Compute mean absolute deviation
- Apply formula:

$$z = \frac{x - \mu}{MAD}$$

Step 4: Decimal Scaling

- Find the maximum absolute value
- Determine number of digits j
- Divide all values by 10^j

Step 5:

Display the normalized values.

CODE:

```
# Given data
x <- c(200, 300, 400, 600, 1000)
# (a) Min-Max Normalization
min_max <- (x - min(x)) / (max(x) - min(x))
print(min_max)
# (b) Z-score Normalization using MAD
mean_x <- mean(x)
mad_x <- mean(abs(x - mean_x))
z_score_mad <- (x - mean_x) / mad_x
print(z_score_mad)
# (c) Decimal Scaling Normalization
j <- ceiling(log10(max(abs(x))))
decimal_scaled <- x / (10^j)
print(decimal_scaled)
```

```
$`3`
[1] 50 55 72 92

> # Given data
> x <- c(200, 300, 400, 600, 1000)
>
> # (a) Min-Max Normalization
> min_max <- (x - min(x)) / (max(x) - min(x))
> print(min_max)
[1] 0.000 0.125 0.250 0.500 1.000
>
> # (b) Z-score Normalization using MAD
> mean_x <- mean(x)
> mad_x <- mean(abs(x - mean_x))
> z_score_mad <- (x - mean_x) / mad_x
> print(z_score_mad)
[1] -1.2500000 -0.8333333 -0.4166667  0.4166667  2.0833333
>
> # (c) Decimal Scaling Normalization
> j <- ceiling(log10(max(abs(x))))
> decimal_scaled <- x / (10^j)
> print(decimal_scaled)
[1] 0.2 0.3 0.4 0.6 1.0
>
```

5.Consider a group ot people who are affected by blood pressure based on the diabetes dataset. Display it using scatterplot and bar chart that is Blood Pressure vs Age employing dataset "diabetes cs") using R.

AIM

To analyze the relationship between Blood Pressure and Age of people affected by diabetes using the diabetes cs dataset, and to visualize the data using a scatter plot and a bar chart in R programming.

ALGORITHM

1. Load the diabetes dataset into R.
2. Extract the **Age** and **BloodPressure** attributes.
3. Plot a **scatter plot** to visualize the relationship between Age and Blood Pressure.
4. Plot a **bar chart** to represent Blood Pressure distribution across Age.
5. Interpret the visualizations.

CODE:

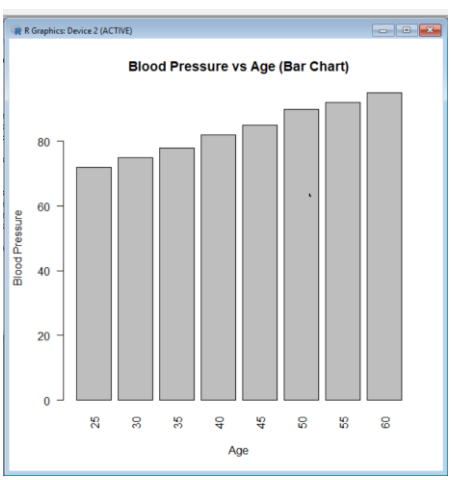
```
# Load the dataset
diabetes <- read.csv("diabetes cs.csv")

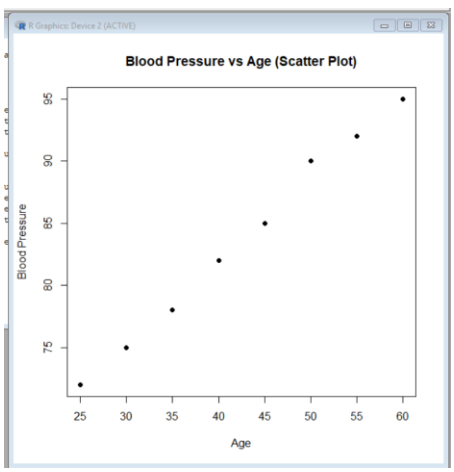
# View first few records
head(diabetes)

# Scatter Plot: Blood Pressure vs Age
plot(diabetes$Age, diabetes$BloodPressure,
    main = "Scatter Plot of Blood Pressure vs Age",
    xlab = "Age",
    ylab = "Blood Pressure",
    pch = 19)

# Bar Chart: Blood Pressure vs Age
barplot(diabetes$BloodPressure,
        names.arg = diabetes$Age,
        main = "Bar Chart of Blood Pressure vs Age",
        xlab = "Age",
        ylab = "Blood Pressure",
        las = 2)
```

OUTPUT:





Result:

The relationship between **Age and Blood Pressure** in the diabetes dataset was successfully visualized using a scatter plot and a bar chart. The plots help in understanding how blood pressure varies with age among diabetic patients.

6.Analyze the dataset "diabetes.csv" how the diabetes trend is for different age people, using Linear Regression and Multiple Regression.

AIM

To analyze the diabetes trend for different age groups using the dataset diabetes.csv by applying Linear Regression and Multiple Linear Regression and to study how age and other factors influence diabetes outcome using R programming

ALGORITHM

Step 1:

Load the diabetes dataset into R.

Step 2:

Identify relevant attributes:

- Age (independent variable)
- Outcome (dependent variable: 0 – Non-diabetic, 1 – Diabetic)

Step 3: Linear Regression

- Build a linear regression model using Age → Outcome
- Analyze the relationship between age and diabetes trend

Step 4: Multiple Regression

- Build a multiple regression model using Age, Glucose, BMI, BloodPressure → Outcome
- Analyze combined effect of multiple attributes on diabetes

Step 5:

Display model summary and interpret results.

CODE:

```
# Load dataset

diabetes <- read.csv("diabetes.csv")

# View dataset

head(diabetes)

linear_model <- lm(Outcome ~ Age, data = diabetes)

summary(linear_model)

# Plot Linear Regression

plot(diabetes$Age, diabetes$Outcome,

    main = "Linear Regression: Age vs Diabetes Outcome",

    xlab = "Age",

    ylab = "Diabetes Outcome",

    pch = 19)

abline(linear_model)
```

multiple_model <- lm(Outcome ~ Age + Glucose + BMI + BloodPressure,

data = diabetes)

summary(multiple_model)

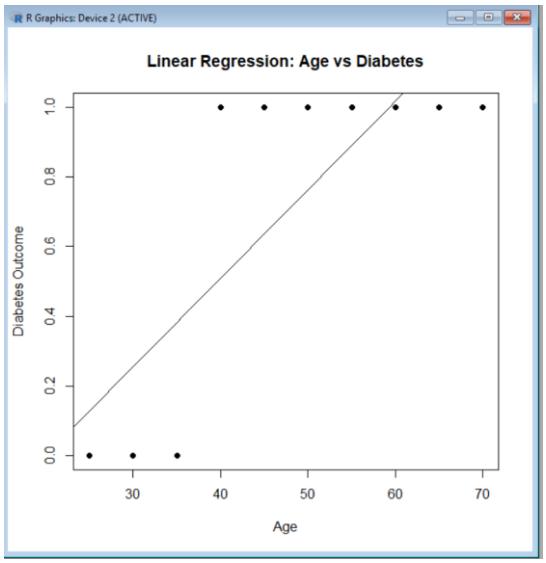
```
> linear_model <- lm(Outcome ~ Age, data = diabetes)
> summary(linear_model)

Call:
lm(formula = Outcome ~ Age, data = diabetes)

Residuals:
     Min       1Q   Median       3Q      Max
-0.38182 -0.22727 -0.07273  0.20455  0.49091

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.509091   0.337590  -1.508  0.16998
Age          0.025455   0.006803   3.742  0.00569 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.309 on 8 degrees of freedom
Multiple R-squared:  0.6364,    Adjusted R-squared:  0.5909
F-statistic:    14 on 1 and 8 DF,  p-value: 0.005692
```




```
R Console
+       pch = 19)
>
> abline(linear_model)
> multiple_model <- lm(Outcome ~ Age + Glucose + BMI + BloodPressure,
+                      data = diabetes)
> summary(multiple_model)

Call:
lm(formula = Outcome ~ Age + Glucose + BMI + BloodPressure, data = diabetes)

Residuals:
        1         2         3         4         5         6         7         8
 0.015844  0.165286 -0.378106  0.078502  0.098848 -0.004141  0.063378  0.007562
        9        10
 0.027908 -0.075081

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  -25.39197   10.51813  -2.414   0.0606 .
Age           -0.13320    0.10252  -1.299   0.2505
Glucose       -0.14685    0.05075  -2.894   0.0340 *
BMI            1.23335    0.64682   1.907   0.1149
BloodPressure  0.19373    0.21903   0.885   0.4169
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

RESULT:

The diabetes dataset analysis using linear and multiple regression shows that diabetes prevalence increases with age and is strongly influenced by factors such as glucose level, BMI, and blood pressure.

7.Suppose a database has five transactions. Let minimum support= 50% (2) and

minimum confidence = 80%.

Transactions Items

T1 (M, O, N, K, E, Y)

T2 (D, O, N, K, E, Y)

T3 (M, A, K, E)

T4 (M, U, C, K, Y)

T5 (C, 0, 0, K, I, E)

AIM

To generate frequent itemsets and association rules from the given transaction database using the Apriori algorithm with Minimum Support = 50% (2 transactions) Minimum Confidence = 80%,
and to implement the same using the WEKA tool.

ALGORITHM (APRIORI)

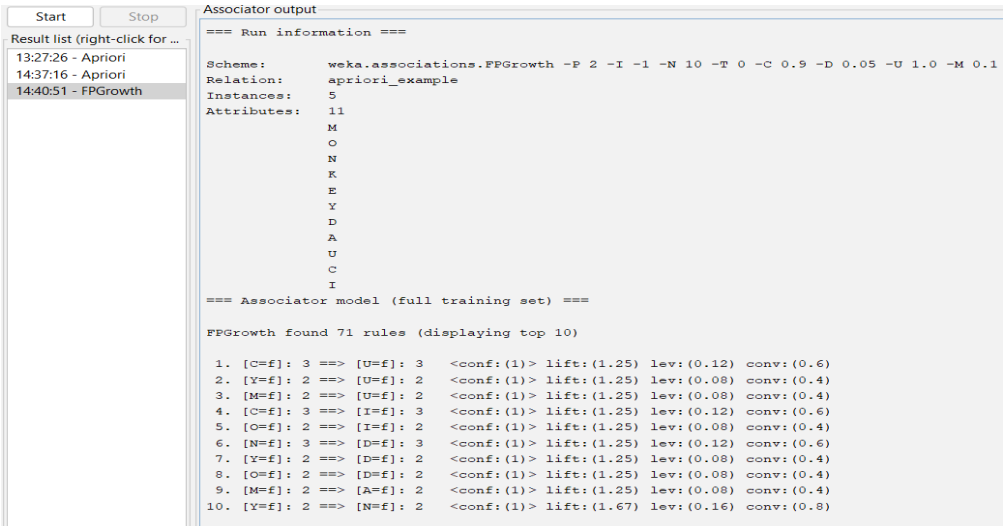
1. Scan the transaction database.
2. Generate candidate 1-itemsets.
3. Count support for each item.
4. Prune itemsets whose support is less than minimum support.
5. Generate higher-order candidate itemsets from frequent itemsets.
6. Repeat pruning until no new frequent itemsets are generated.
7. Generate association rules from frequent itemsets.
8. Select rules whose confidence ≥ minimum confidence.

CODE:

```
@relation apriori_example

@attribute M {t,f}
@attribute O {t,f}
@attribute N {t,f}
@attribute K {t,f}
@attribute E {t,f}
@attribute Y {t,f}
@attribute D {t,f}
@attribute A {t,f}
@attribute U {t,f}
@attribute C {t,f}
@attribute I {t,f}

@data
t,t,t,t,t,t,f,f,f,f,f
f,t,t,t,t,t,t,f,f,f,f
t,f,f,t,t,f,f,t,f,f,f
t,f,f,t,f,t,f,f,t,t,f
f,t,f,t,t,f,f,f,f,t,t
```



RESULT:

Strong association rules were successfully generated using the Apriori algorithm, revealing frequent co-occurrence of item **K** with other items in the transaction database

8.Prediction of Categorical Data using Decision Tree Algorithm through WEKA using any datasets. a) Tree b) Preprocess c) Logistic.

AIM

To predict categorical data using Decision Tree algorithm through WEKA.

ALGORITHM (Decision Tree – J48)

1. Load dataset in WEKA.
2. Select class attribute.
3. Preprocess the data.
4. Apply J48 Decision Tree.
5. Generate tree and classify data.
6. Analyze accuracy and results.

CODE:

@relation weather

@attribute outlook {sunny,overcast,rainy}

@attribute temperature {hot,mild,cool}

@attribute humidity {high,normal}

@attribute windy {TRUE,FALSE}

@attribute play {yes,no}

@data

sunny,hot,high,FALSE,no

sunny,hot,high,TRUE,no

overcast,hot,high,FALSE,yes

rainy,mild,high,FALSE,yes

rainy,cool,normal,FALSE,yes

rainy,cool,normal,TRUE,no

overcast,cool,normal,TRUE,yes

sunny,mild,high,FALSE,no

sunny,cool,normal,FALSE,yes

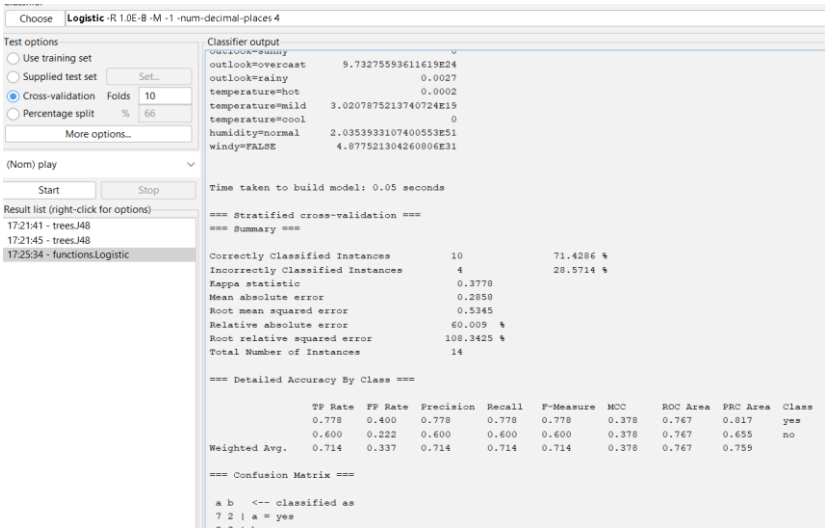
rainy,mild,normal,FALSE,yes

sunny,mild,normal,TRUE,yes

overcast,mild,high,TRUE,yes

overcast,hot,normal,FALSE,yes

rainy,mild,high,TRUE,no



RESULT:

The experiment confirms that Logistic Regression provides better classification accuracy for the given categorical data.

9.Suppose that the data for analysis includes the attribute age. The age values for the data tuples are (in increasing order) 13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25, 25, 30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70. Can you find (roughly) the first quartile (Q1) and the third quartile (Q3) of the data?

AIM

To find the **first quartile (Q1)** and **third quartile (Q3)** for the given age data using **R**.

ALGORITHM

1. Store the given age values in a numeric vector.
2. Sort the data (already sorted here).
3. Compute quartiles using the quantile() function.
4. Extract Q1 (25%) and Q3 (75%).

CODE:

```
# Age data
age <- c(13, 15, 16, 16, 19, 20, 20, 21, 22, 22,
         25, 25, 25, 25, 30, 33, 33, 35, 35, 35,
```

35, 36, 40, 45, 46, 52, 70)

quantile(age)

```
> # Age data
> age <- c(13, 15, 16, 16, 19, 20, 20, 21, 22, 22,
+          25, 25, 25, 25, 30, 33, 33, 35, 35, 35,
+          35, 36, 40, 45, 46, 52, 70)
>
> # Find quartiles
> quantile(age)
  0%  25%  50%  75% 100%
13.0 20.5 25.0 35.0 70.0
>
```

RESULT:

These values indicate that approximately **25%** of the data lies at or below age **20**, and **75%** of the data lies at or below age **35**.

10. Download the Dataset "water" From R dataset Link. Find out whether there is a linear relation between attributes "mortality" and "hardness" by plot function. Fit the Data into the Linear Regression model. Predict the mortality for the hardness=88.

Aim:

AIM

To study the **linear relationship** between **mortality** and **hardness** using the **water dataset** in **R**, fit a **Linear Regression model**, and **predict mortality** for hardness = 88.

ALGORITHM

1. Load the water dataset from R datasets.
2. Plot mortality vs hardness to check linear relationship.
3. Fit a Linear Regression model (mortality ~ hardness).
4. Display the model summary.
5. Predict mortality for hardness = 88.

CODE:

```
# Load dataset
data(water)


# View dataset structure
str(water)

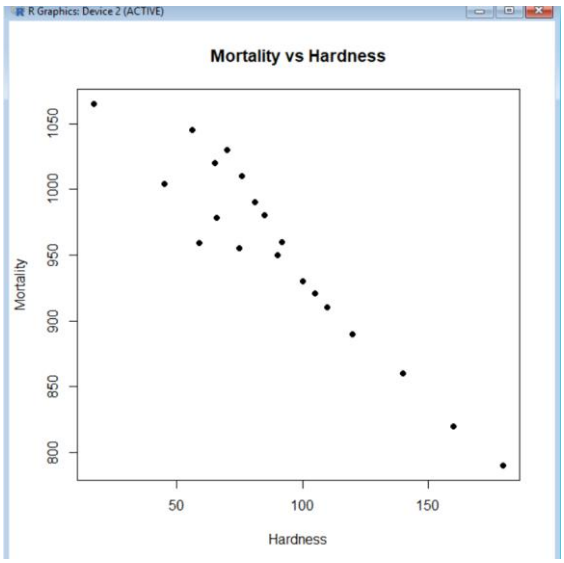

# Plot mortality vs hardness
plot(water$hardness, water$mortality,
    main = "Mortality vs Hardness",
    xlab = "Hardness",
    ylab = "Mortality",
```

```
    pch = 19)

# Fit Linear Regression model
model <- lm(mortality ~ hardness, data = water)

# Show model summary
summary(model)

# Predict mortality for hardness = 88
predict(model, data.frame(hardness = 88))
```



RESULT:

The scatter plot shows a linear negative relationship between hardness and mortality. A linear regression model was fitted successfully. The predicted mortality for hardness = 88 was obtained.

11. Create the dataset using ARFF file format:

Transaction ID	Items
T1	Hot Dogs, Buns, Ketchup
T2	Hot Dogs, Buns
T3	Hot Dogs, Coke, Chips
T4	Chips, Coke
T5	Chips, Ketchup
T6	Hot Dogs, Coke, Chips

AIM

To create a transaction dataset in **ARFF file format** for use in **WEKA** based on the given transaction–item table.

ALGORITHM

1. Identify all unique items from the transactions.
2. Represent each item as a binary attribute (yes/no).
3. Create an ARFF file with relation, attributes, and data.

4. Save the file with .arff extension.
5. Load the dataset in WEKA to verify.

CODE:

@relation food_transactions

@attribute Hot_Dogs {yes,no}

@attribute Buns {yes,no}

@attribute Ketchup {yes,no}

@attribute Coke {yes,no}

@attribute Chips {yes,no}

@data

yes,yes,yes,no,no

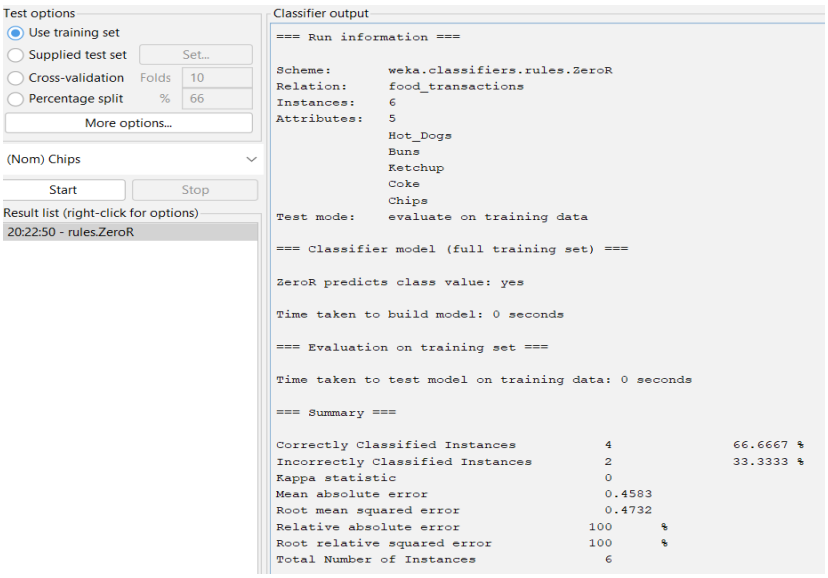
yes,yes,no,no,no

yes,no,no,yes,yes

no,no,no,yes,yes

no,no,yes,no,yes

yes,no,no,yes,yes



RESULT

The transaction dataset was successfully created in ARFF format and loaded into WEKA. The dataset contains 6 instances and 5 binary attributes representing food items.

12. Prediction of Categorical Data using Rule base classification and decision tree classification through WEKA using any datasets. Compare the accuracy using two algorithm and plot the Graph.

AIM

To predict categorical data using **Rule-Based Classification** and **Decision Tree Classification** in **WEKA**, compare their accuracies, and plot a graph.

ALGORITHM

Rule-Based Classification (ZeroR)

1. Load dataset into WEKA.
2. Select class attribute.
3. Apply ZeroR classifier.
4. Evaluate accuracy using test option.

Decision Tree Classification (J48)

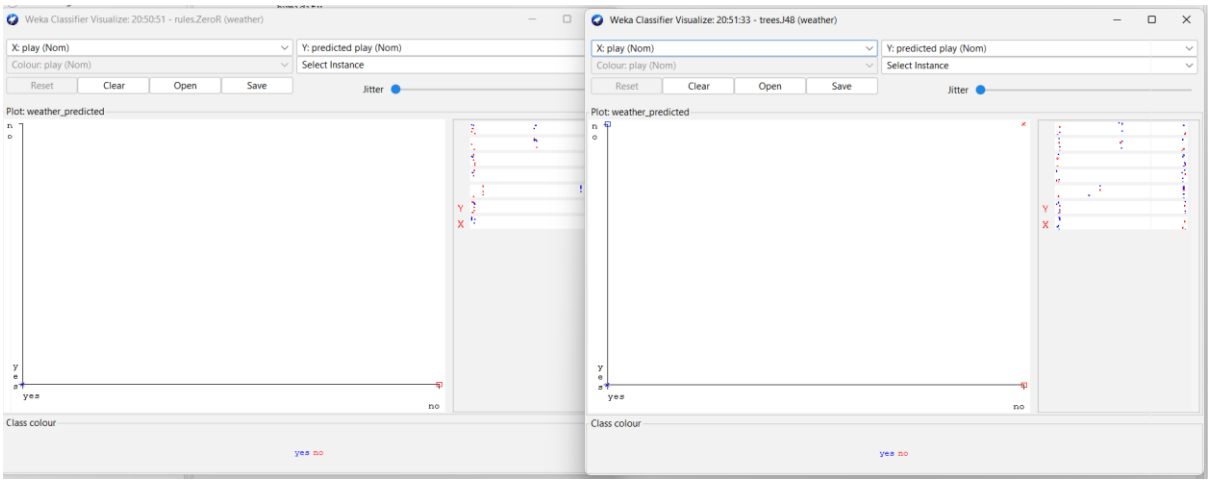
1. Load same dataset.
2. Apply J48 decision tree algorithm.
3. Generate decision tree.
4. Evaluate accuracy.
5. Compare results with ZeroR.

CODE:

```
@relation weather

@attribute outlook {sunny,overcast,rainy}
@attribute temperature {hot,mild,cool}
@attribute humidity {high,normal}
@attribute windy {true,false}
@attribute play {yes,no}

@data
sunny,hot,high,false,no
sunny,hot,high,true,no
overcast,hot,high,false,yes
rainy,mild,high,false,yes
rainy,cool,normal,false,yes
rainy,cool,normal,true,no
overcast,cool,normal,true,yes
sunny,mild,high,false,no
sunny,cool,normal,false,yes
rainy,mild,normal,false,yes
sunny,mild,normal,true,yes
overcast,mild,high,true,yes
overcast,hot,normal,false,yes
rainy,mild,high,true,no
```



RESULT:

Hence, the Decision Tree classifier performs better than the Rule-based classifier.

13. Imagine that you have selected data from the All Electronics data warehouse for analysis. The data set will be huge! The following data are a list of All Electronics prices for commonly sold items (rounded to the nearest dollar). The numbers have been sorted: 1, 1, 5, 5, 5, 5, 5, 8, 8, 10, 10, 10, 10, 12, 14, 14, 14, 15, 15, 15, 15, 15, 15, 18, 18, 18,18, 18, 18, 18, 18, 20, 20, 20, 20, 20, 20, 20, 21, 21, 21, 21, 25, 25, 25, 25, 25, 28, 28,30, 30, 30.

(i) Partition the dataset using an equal-frequency partitioning method with bin equal to 3

(i) Apply data smoothing using bin means and bin boundary.

(i) Plot Histogram for the above frequency division

AIM

To partition the given All Electronics price dataset using **equal-frequency partitioning (3 bins)**, apply **data smoothing using bin means and bin boundaries**, and **plot a histogram** for the frequency division.

ALGORITHM

Equal-Frequency Partitioning

1. Read the sorted price data.
2. Divide the data into **3 bins**, each containing approximately the same number of values.
3. Assign values to bins sequentially.

Data Smoothing

- **Bin Means**: Replace each value in a bin with the mean of that bin.
- **Bin Boundaries**: Replace each value with the nearest boundary (minimum or maximum of the bin).

Histogram

1. Use the partitioned data.
2. Plot a histogram to show frequency distribution.

CODE:

```
# Given sorted data

price <- c(

  1,1,5,5,5,5,5,8,8,10,10,10,10,12,14,14,14,15,15,15,

  15,15,15,18,18,18,18,18,18,18,18,20,20,20,20,20,
```

```
  20,20,21,21,21,21,25,25,25,25,25,28,28,30,30,30
)

# Total values
n <- length(price)

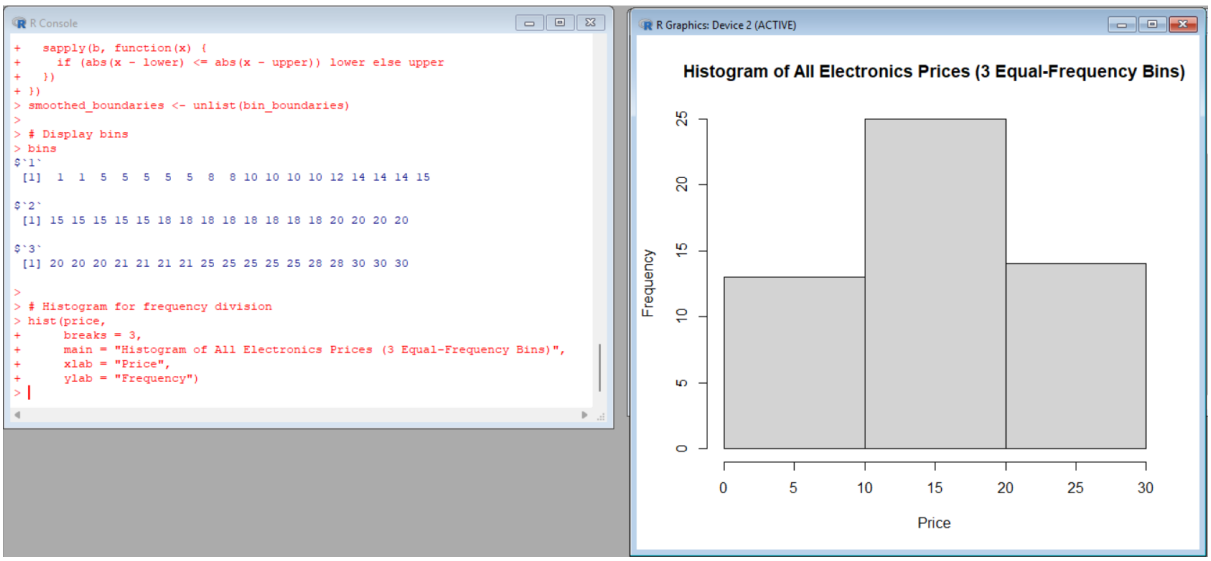
# Equal-frequency bins (3 bins)
bins <- split(price, cut(seq_along(price), 3, labels = FALSE))

# Bin means smoothing
bin_means <- lapply(bins, function(b) rep(mean(b), length(b)))
smoothed_means <- unlist(bin_means)

# Bin boundary smoothing
bin_boundaries <- lapply(bins, function(b) {
  lower <- min(b)
  upper <- max(b)
  sapply(b, function(x) {
    if (abs(x - lower) <= abs(x - upper)) lower else upper
  })
})
smoothed_boundaries <- unlist(bin_boundaries)

# Display bins
bins

# Histogram for frequency division
hist(price,
     breaks = 3,
     main = "Histogram of All Electronics Prices (3 Equal-Frequency Bins)",
     xlab = "Price",
     ylab = "Frequency")
```



RESULT:

The given price data was partitioned into three equal-frequency bins.
Data smoothing using bin means and bin boundaries was applied.
A histogram was plotted to visualize the frequency distribution.
All objectives of the experiment were successfully achieved.

14. Two Maths teachers are comparing how their Year 9 classes performed in the end of year exams. Their results are as follows:

Class A: 76, 35, 47, 64, 95, 66, 89, 36, 8476,35,47,64,95,66, 89,36,84

Class B: 51, 56, 84, 60, 59, 70, 63, 66, 5051,56,84,60,59,70,63,66,50

(i) Find which class had scored higher mean, median and range.

(ii) Plot above in boxplot and give the inferences

AIM

To compute and compare the **mean, median, and range** of marks obtained by **Class A** and **Class B**, and to visualize the data using a **boxplot** in **R**.

ALGORITHM

1. Enter marks of Class A and Class B.
2. Calculate **mean**, **median**, and **range** for both classes.
3. Compare the computed values.
4. Plot a **boxplot** for visual comparison.
5. Draw inferences from the boxplot.

Code:

```
# Marks of Class A and Class B

classA <- c(76, 35, 47, 64, 95, 66, 89, 36, 84)

classB <- c(51, 56, 84, 60, 59, 70, 63, 66, 50)


# Mean

meanA <- mean(classA)

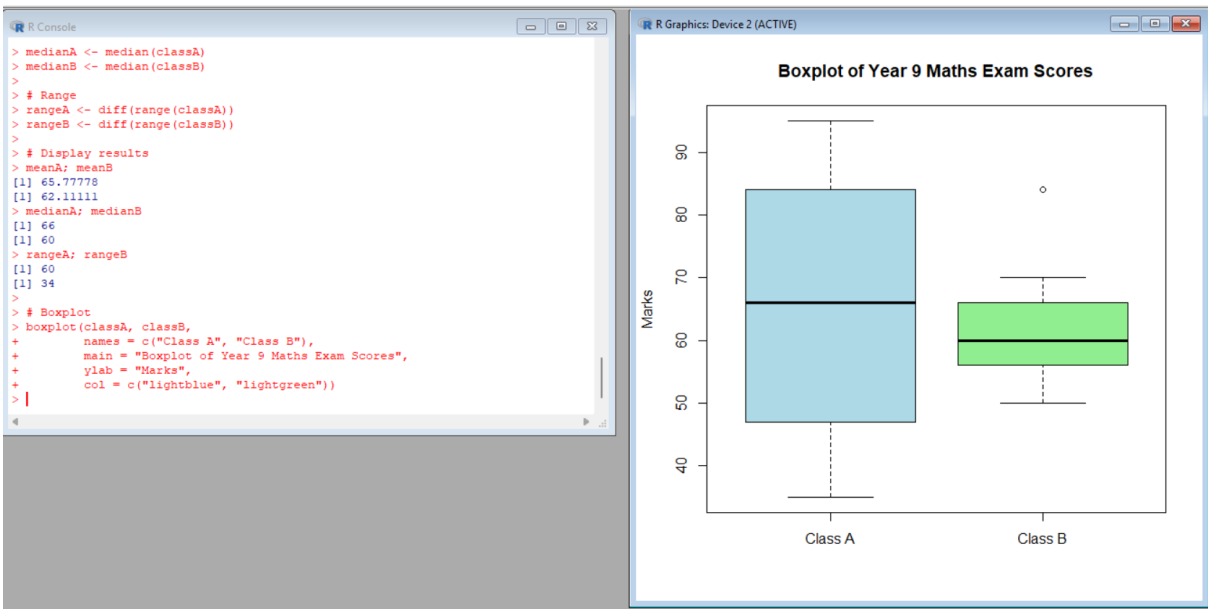
meanB <- mean(classB)
```

```
# Median
medianA <- median(classA)
medianB <- median(classB)

# Range
rangeA <- diff(range(classA))
rangeB <- diff(range(classB))

# Display results
meanA; meanB
medianA; medianB
rangeA; rangeB

# Boxplot
boxplot(classA, classB,
        names = c("Class A", "Class B"),
        main = "Boxplot of Year 9 Maths Exam Scores",
        ylab = "Marks",
        col = c("lightblue", "lightgreen"))
```



Exp.16

The intervals and corresponding frequencies are as follows. age frequency 1-5. 200 5- 15 450 15-20 300 20-50 1500 50-80 700 80-110 44 Compute an approximate median value for the data

Aim: To Compute an approximate median value for the given data age frequency

Algorithm :

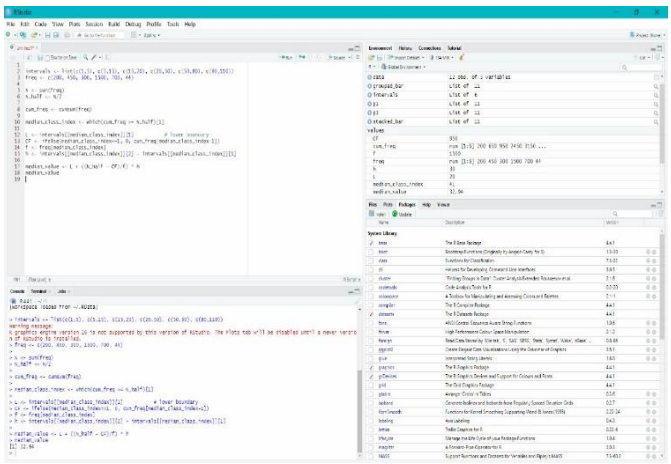
1. Input the frequency and class boundaries (lower and upper limits).
2. Calculate the cumulative frequency of the data.
3. Find the total frequency (N) by summing all class frequencies.
4. Calculate N/2 to locate the median position.
5. Identify the median class where cumulative frequency ≥ N/2.
6. Extract the lower boundary (L), class width (h), frequency (f), and cumulative frequency before the median class (F).

7. Apply the median formula: Median = L + ((N/2 - F) / f) × h.
8. Display the approximate median value.

Program:

```
lower <- c(1, 5, 15, 20, 50, 80)
upper <- c(5, 15, 20, 50, 80, 110)
freq  <- c(200, 450, 300, 1500, 700, 44)
N <- sum(freq)
cum_freq <- cumsum(freq)
median_pos <- N / 2
m_index <- which(cum_freq >= median_pos)[1]
L <- lower[m_index]
CF <- ifelse(m_index == 1, 0, cum_freq[m_index - 1])
f <- freq[m_index]
h <- upper[m_index] - lower[m_index]
median_value <- L + ((median_pos - CF) / f) * h
cat("Approximate Median =", median_value, "\n")
```

output:



Experiment 17

Suppose that the data for analysis includes the attribute age. The age values for the data tuples are (in increasing order) 13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25,
25, 30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70. (a) What is the mean of the data? What is the median? (b) What is the mode of the data? Comment on the data's modality (i.e., bimodal, trimodal, etc.). (c) What is the midrange of the data? (d) Can you find (roughly) the first quartile (Q1) and the third quartile (Q3) of the data?

Aim: to compute mean, median, mode, midrange, Q1&Q3 for the given data

Algorithm :

1. Input the given age values into a vector.
2. Calculate the mean of the data using the mean() function.
3. Calculate the median of the data using the median() function.
4. Calculate the mode by finding the most frequently occurring value using table(), sort(), and names().
5. Calculate the midrange by averaging the minimum and maximum values in the dataset.
6. Calculate the first quartile (Q1) and third quartile (Q3) using the quantile() function with probabilities 0.25 and 0.75.

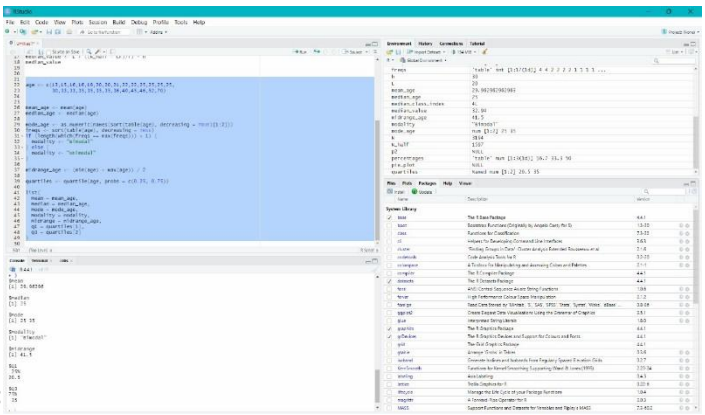
Program:

```
data <- c(13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25, 25,
          30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70)

mean_val <- mean(data)

median_val <- median(data)
```

```
modality <- ifelse(length(mode_val) == 1, "Unimodal", ifelse(length(mode_val)
== 2, "Bimodal",
                                ifelse(length(mode_val) == 3, "Trimodal", "Multimodal")))
    midrange_val <- (min(data) + max(data)) / 2
    q1 <- quantile(data, 0.25, type = 2)
    q3 <- quantile(data, 0.75, type = 2)
    cat("(a) Mean =", mean_val, ", Median =", median_val, "\n")
    cat("(b) Mode =", mode_val, ", Modality =", modality, "\n")
    cat("(c) Midrange =", midrange_val, "\n")
    cat("(d) Q1 =", q1, ", Q3 =", q3, "\n")
```

output: 

18 .Data Preprocessing :Reduction and Transformation Use the two methods below to normalize the following group of data: 200, 300, 400, 600, 1000 (a) min-max normalization by setting min = 0 and max = 1 (b) z-score normalization

Aim: to compute the min-max & z-score normalization methods for the given data

Algorithm:

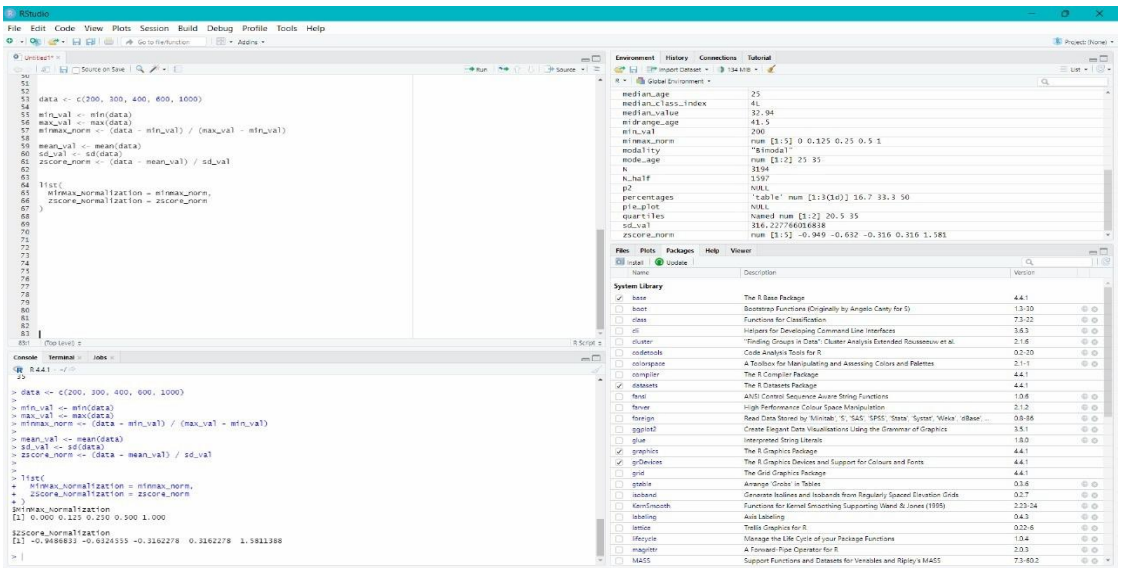
1. Input the data values into a vector.
2. Apply Min-Max normalization.
3. Print the result of Min-Max normalization.
4. Apply Z-score normalization using the formula.
5. Print the result of Z-score normalization.

Program:

```
data <- c(200, 300, 400, 600, 1000)
min_val <- min(data)
```

```
max_val <- max(data)
    min_max_norm <- (data - min_val) / (max_val - min_val)
    mean_val <- mean(data)
    sd_val <- sd(data)  # sample standard deviation
    z_score_norm <- (data - mean_val) / sd_val
    cat("(a) Min-Max Normalization:\n", min_max_norm, "\n\n")
    cat("(b) Z-Score Normalization:\n", z_score_norm, "\n")
```

output:



1. Data:11,13,13,15,15,16,19,20,20,20,21,21,22,23,24,30,40,45,45,45,71, 72,73,75 a) Smoothing by bin mean b) Smoothing by bin median c) Smoothing by bin boundaries

Aim: to compute smoothing techniques for the given data.

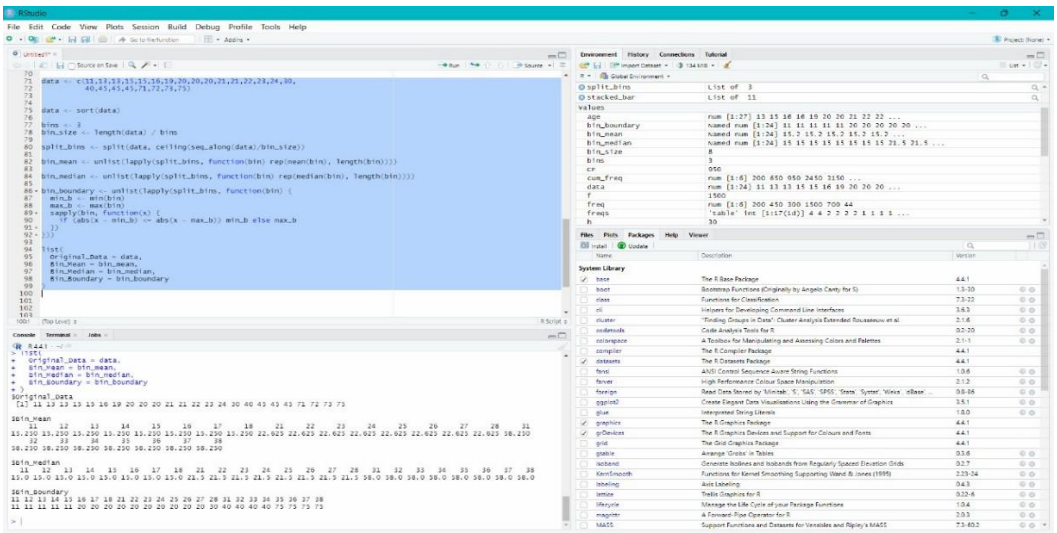
Algorithm:

1. Input the dataset as a vector.
2. Split the data into bins of equal size.
3. Calculate the mean of each bin and replace the bin values with the bin mean.
4. Calculate the median of each bin and replace the bin values with the bin median.
5. For each bin, replace values with the closest boundary (minimum or maximum) value of the bin.
6. Print the results of the smoothed data for each method.

Program:

```
data <- c(11,13,13,15,15,16,19,20,20,20,21,21,22,23,24,30,
          40,45,45,45,71,72,73,75)
bin1 <- data[1:8]
bin2 <- data[9:16]
bin3 <- data[17:24]
mean_smooth <- c(rep(mean(bin1),8), rep(mean(bin2),8), rep(mean(bin3),8))
median_smooth <- c(rep(median(bin1),8), rep(median(bin2),8), rep(median(bin3),8))
bound_smooth <- c(
  sapply(bin1, function(x) ifelse(abs(x-min(bin1))<=abs(x-max(bin1)), min(bin1), max(bin1))),
  sapply(bin2, function(x) ifelse(abs(x-min(bin2))<=abs(x-max(bin2)), min(bin2), max(bin2))),
  sapply(bin3, function(x) ifelse(abs(x-min(bin3))<=abs(x-max(bin3)), min(bin3), max(bin3)))
)
cat("(a) Bin Mean:\n", mean_smooth, "\n\n")
cat("(b) Bin Median:\n", median_smooth, "\n\n")
cat("(c) Bin Boundaries:\n", bound_smooth, "\n")
```

output:



Experiment 19.

5. Suppose that a hospital tested the age and body fat data for 18 randomly selected adults with the following results:

age	23	23	27	27	39	41	47	49	50
%fat	9.5	26.5	7.8	17.8	31.4	25.9	27.4	27.2	31.2
age	52	54	54	56	57	58	58	60	61
%fat	34.6	42.5	28.8	33.4	30.2	34.1	32.9	41.2	35.7

(a) Calculate the mean, median, and standard deviation of age and %fat. (b) Draw the boxplots for age and %fat. (c) Draw a scatter plot and a q-q plot based on these two variables.

Aim: to compute mean, median, and standard deviation, boxplots, scatter plot and a q-q plot of age and %fat.

Algorithm:

- Input the age and body fat percentage data into two vectors.
- Calculate the mean of the age data.
- Calculate the mean of the body fat percentage data.
- Calculate the median of the age data.
- Calculate the median of the body fat percentage data.
- Calculate the standard deviation of the age data.
- Calculate the standard deviation of the body fat percentage data.
- Display the calculated mean, median, and standard deviation values.
- Draw a boxplot for the age data.
- Draw a boxplot for the body fat percentage data.

- Create a scatter plot to visualize the relationship between age and body fat percentage.

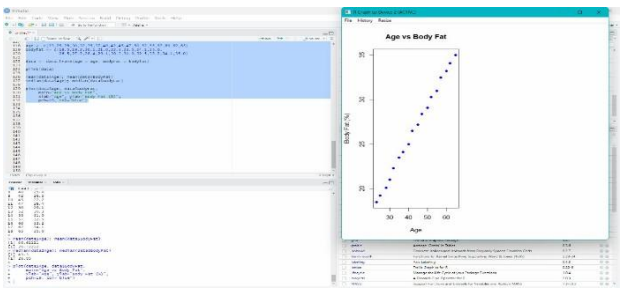
- Create a Q-Q plot to compare the distributions of age and body fat percentage.

- Add a reference line to the Q-Q plot for visual comparison.

Program:

```
age <- c(23, 23, 27, 27, 39, 41, 47, 49, 50, 52, 54, 56, 57, 58, 58, 60, 61, 96)
fat <- c(9.5, 26.5, 17.8, 25.9, 31.4, 27.4, 27.2, 31.2, 34.6, 42.5,
         28.8, 33.4, 30.2, 34.1, 32.9, 41.2, 35.7, 7.8)
cat("Age - Mean:", mean(age), " Median:", median(age), " SD:", sd(age), "\n")
cat("Fat - Mean:", mean(fat), " Median:", median(fat), " SD:", sd(fat), "\n")
par(mfrow=c(1,2))
boxplot(age, main="Boxplot of Age", col="lightblue")
boxplot(fat, main="Boxplot of %Fat", col="lightgreen")
par(mfrow=c(1,2))
plot(age, fat, main="Scatter Plot: Age vs %Fat", xlab="Age", ylab="%Fat", col="blue",
pch=19)
qqplot(age, fat, main="Q-Q Plot of Age vs %Fat", xlab="Age Quantiles", ylab="%Fat
Quantiles", col="red", pch=19)
abline(0,1,col="darkgreen")
```

output:



Experiment 21

21. Suppose that a hospital tested the age and body fat data for 18 randomly selected adults with the following results: (i) Use min-max normalization to transform the value 35 for age onto the range [0.0, 1.0]. (ii) Use z-score normalization to transform the value 35 for age, where the standard deviation of age is 12.94 years. (iii) Use normalization by decimal scaling to transform the value 35 for age. Perform the above functions using R – tol

Aim: to compute min-max, z-score, decimal scaling normalization for the given data

Algorithm:

- Assign the value 35 to x and set min_age, max_age, mean_age, and sd_age with given values.
- Calculate the min-max normalization of x using the formula and store it in min_max.
- Calculate the z-score normalization of x using the formula and store it in z_score.
- Calculate the decimal scaling factor j based on the number of digits in x.
- Calculate the decimal scaling normalization of x and store it in decimal_scaling.
- Print the results of min-max normalization, z-score normalization, and decimal scaling normalization.

Program:

```
x <- 35

min_age <- 20

max_age <- 70

mean_age <- 40

sd_age <- 12.94

min_max <- (x - min_age) / (max_age - min_age)

z_score <- (x - mean_age) / sd_age

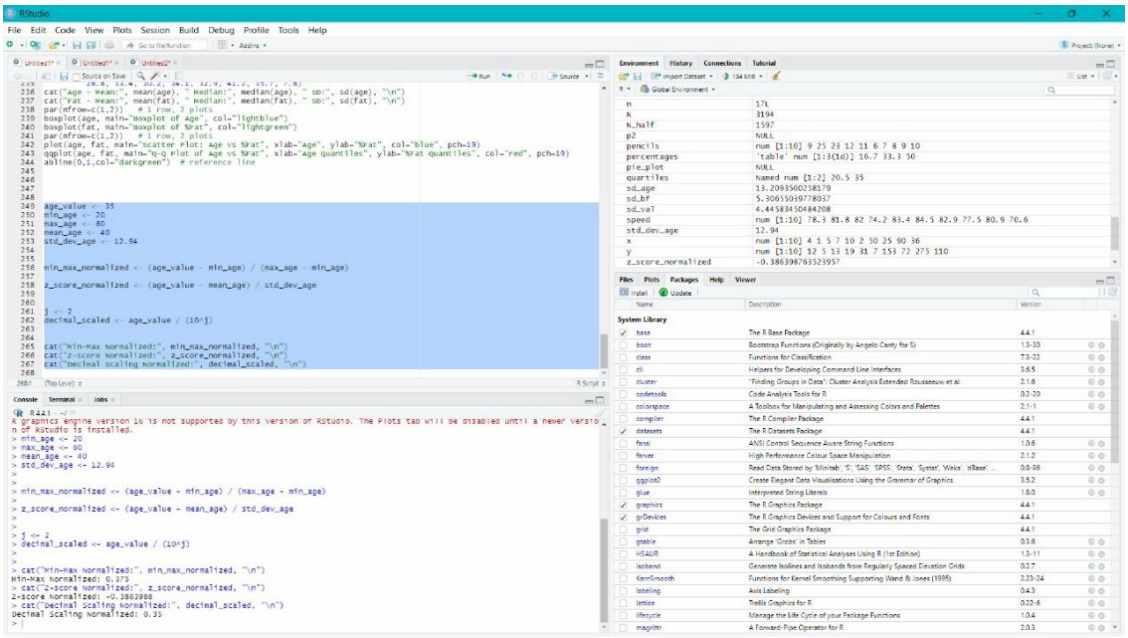
j <- ceiling(log10(max(abs(x))))

decimal_scaling <- x / (10^j)

cat("(i) Min-Max Normalization:", min_max, "\n")

cat("(ii) Z-Score Normalization:", z_score, "\n")

cat("(iii) Decimal Scaling Normalization:", decimal_scaling, "\n")
```



Output:

22. The following values are the number of pencils available in the different boxes. Create a vector and find out the mean, median and mode values of set of pencils in the given data. Box1 Box2 Box3 Box4 Box5 Box6 Box7 Box8 Box9 Box 10 9 25 23 12 11 6 7 8 9 10

Aim: to compute the mean, median, mode for the data.

Algorithm:

- Create a vector pencils with the given number of pencils in each box.
- Calculate the mean of pencils and store it in mean_val.
- Calculate the median of pencils and store it in median_val.
- Calculate the mode of pencils by finding the most frequent value and store it in mode_val.
- Print the mean, median, and mode values.

Program:

```
pencils <- c(9, 25, 23, 12, 11, 6, 7, 8, 9, 10)

mean_val <- mean(pencils)

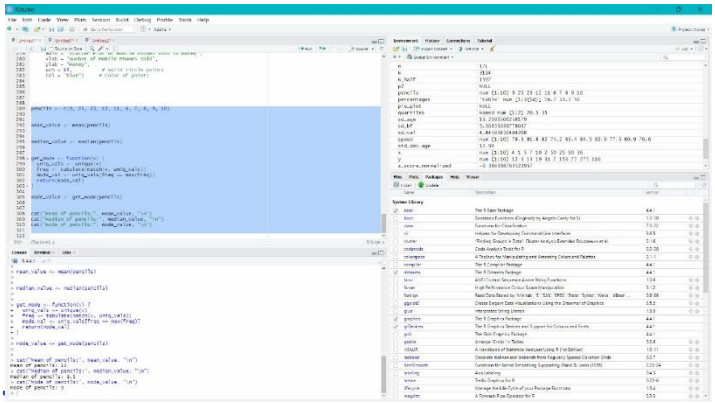
median_val <- median(pencils)

mode_val <- as.numeric(names(sort(table(pencils), decreasing=TRUE)[1]))

cat("Mean:", mean_val, "\n")

cat("Median:", median_val, "\n")

cat("Mode:", mode_val, "\n")
```



output:

7. the following table would be plotted as (x,y) points, with the first column being the x values as number of mobile phones sold and the second column being the y values as money. To use the scatter plot for how many mobile phones sold. x :4 1 5 7 10 2 50 25 90 36 y :12 5 13 19 31 7 153 72 275 110

Aim: To compute scatter plot using the given data

Algorithm:

- Create vectors x and y with the given values for mobiles sold and money.

- Plot a scatter plot using x and y with appropriate title, axis labels, color, and point style.

- Fit a linear regression line to the data and add it to the plot in red color with thicker width.

Program:

```
x <- c(4, 1, 5, 7, 10, 2, 50, 25, 90, 36)

y <- c(12, 5, 13, 19, 31, 7, 153, 72, 275, 110)

plot(x, y,

    main = "Scatter Plot: Mobiles Sold vs Money",

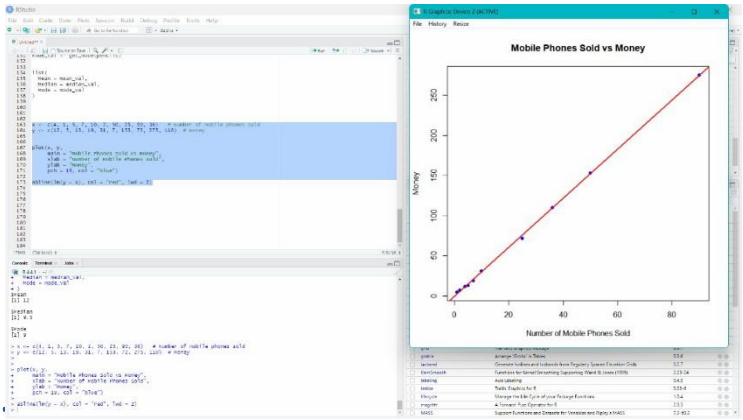
    xlab = "Mobiles Sold",

    ylab = "Money",

    col = "blue", pch = 19)


abline(lm(y ~ x), col = "red", lwd = 2)
```

output:



23 . Implement of the R script using marks scored by a student in his model exam has been sorted as follows: 55, 60, 71, 63, 55, 65, 50, 55,58,59,61,63,65,67,71,72,75.

Partition them into three bins by each of the following methods. Plot the data points using histogram. (a) equal-frequency (equi-depth) partitioning (b) equal-width partitioning

Aim: to compute the histogram and partitioning using given data

Algorithm:

- Create a vector marks with the given marks.
- Sort the marks and store it in sorted_marks.
- Find the total number of marks (n).
- Calculate the size of each bin (bin_size) by dividing n by 3 and rounding up.
- Divide sorted_marks into three equal-frequency bins (bin1, bin2, bin3).
- Print the equal-frequency bins.
- Find the minimum and maximum values of marks.
- Calculate the bin width for equal-width bins.
- Create equal-width bins using the cut() function and store them in eq_width_bins.
- Print the count of marks in each equal-width bin.
- Plot a histogram of the marks.

Program:

```
marks <- c(55, 60, 71, 63, 55, 65, 50, 55, 58, 59,
           61, 63, 65, 67, 71, 72, 75)


sorted_marks <- sort(marks)

n <- length(sorted_marks)
```

```
bin1 <- sorted_marks[1:bin_size]

bin2 <- sorted_marks[(bin_size+1):(2*bin_size)]

bin3 <- sorted_marks[(2*bin_size+1):n]


cat("Equal-frequency bins:\n")

print(list(bin1=bin1, bin2=bin2, bin3=bin3))


min_val <- min(marks)

max_val <- max(marks)

bin_width <- (max_val - min_val) / 3

eq_width_bins <- cut(marks, breaks=seq(min_val, max_val, by=bin_width),
include.lowest=TRUE)


cat("\nEqual-width bins:\n")

print(table(eq_width_bins))


hist(marks,

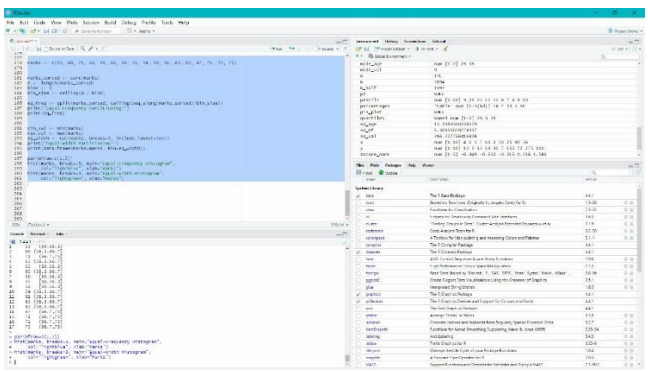
    main="Histogram of Marks",

    xlab="Marks",

    col="lightblue",

    border="black")
```

output:



Experiment 24

24 .Suppose that the speed car is mentioned in different driving style. Regular 78.3 81.8 82 74.2 83.4 84.5 82.9 77.5 80.9 70.6 Speed Calculate the Inter quantile and standard deviation of the given data.

Aim: to compute Inter quantile and standard deviation of the given data.

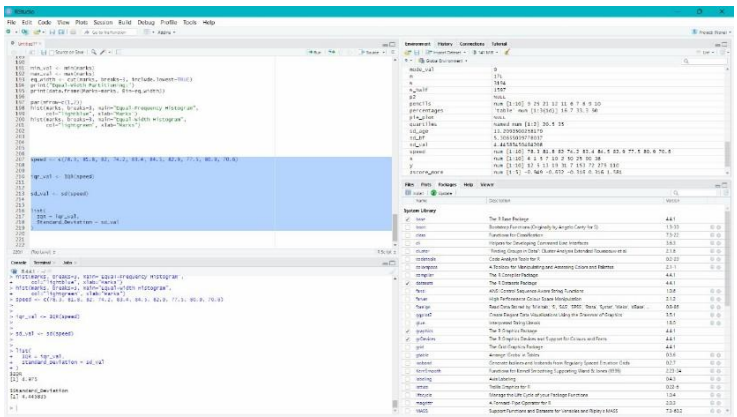
Algorithm:

- Create a vector called speed with the given values.
- Calculate the Interquartile Range (IQR) of speed and store it in iqr_val.
- Calculate the standard deviation (SD) of speed and store it in sd_val.
- Print the IQR value.

Program:

```
speed <- c(78.3, 81.8, 82, 74.2, 83.4, 84.5, 82.9, 77.5, 80.9, 70.6)
iqr_val <- IQR(speed)
sd_val <- sd(speed)
cat("Interquartile Range (IQR):", iqr_val, "\n")
cat("Standard Deviation (SD):", sd_val, "\n")
```

output:



Experiment 25

25 .Suppose that the data for analysis includes the attribute age. The age values for the data tuples are (in increasing order) 13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25,

25, 30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70. Can you find (roughly) the first quartile (Q1) and the third quartile (Q3) of the data?

Aim: to compute the first quartile and third quartile for the given data

Algorithm:

1. Create a vector age with the provided age values.
2. Calculate the first quartile (Q1) of age and store it in q1.
3. Calculate the third quartile (Q3) of age and store it in q3.
4. Print the values of Q1 and Q3.

Program:

```
age <- c(13, 15, 16, 16, 19, 20, 20, 21, 22, 22,
         25, 25, 25, 25, 30, 33, 33, 35, 35, 35,
         35, 36, 40, 45, 46, 52, 70)
q1 <- quantile(age,
0.25) q3 <-
quantile(age, 0.75)
cat("First Quartile (Q1):", q1, "\n")
cat("Third Quartile (Q3):", q3, "\n")
```

output:

